In terms of signals (and sources), we can differentiate between two cases:

- Sources, such as code, that you own and can instrument yourself (that is, decide where and what kind of signals to emit).
- Sources you don't own, meaning any signals will be predefined. Your task is limited to selecting which signals to consume, in this case. This might be due to several reasons:
  - Code that is a dependency of your code (library, package, module, etc.).
  - The component's source code is not available (e.g., a binary file).
  - The component only offers a (networked) API. Any managed service from your cloud provider of choice falls into this category.

No matter whether you own the code and can decide what signal types to emit or are working with a component you don't own, at some point in time, you want to collect the signals and send them to a central place for processing and storage. This is the telemetry part.

With observability, you want to take an approach driven by return on investment (ROI). In other words, you not only need to understand the different signals available but also the costs of each signal type and what they enable you to do.

In this chapter, we will review the most common signal types, examine how to instrument and collect each, and discuss the costs and benefits of doing so.

2.1 Reference example

While there are several signals to consider, in practice, the three most commonly used are logs, metrics, and traces. In this chapter, we will use a running reference example, a simple Go program called the `echo` service, exposing an HTTP interface, to see the three common signal types in action.

Change to the `ch02/base/` directory, where you find the `main.go` program shown in listing 2.1. As we progress, we will instrument our reference example with different signals with the goal to consume and interact with the signals using open source tooling, including Grafana, Prometheus, and Jaeger.

Listing 2.1 Example Go program to be instrumented

```
package main

import (
    "fmt"
    "log"
    "math/rand"
    "net/http"
    "time"
)

func handleEcho(w http.ResponseWriter, r *http.Request) {
    message := r.URL.Query().Get("message")
    if rand.Intn(100) > 90 {
```

Retrieves the payload to echo from the query parameter message, with the result of being able to interact with the service by issuing an HTTP GET to /echo?message

Simulates a 10% error case